

Deep Bilinear Koopman Model Predictive Control for Nonlinear Dynamical Systems

Dongdong Zhao , Member, IEEE, Boyu Li , Fuxiang Lu , Jinhua She , Fellow, IEEE, and Shi Yan , Member, IEEE

Abstract—This article presents a deep bilinear Koopman model predictive control (DBKMPC) approach for modelling and control of unknown nonlinear systems. The bilinear model, which has the computational speed of a linear model and the predictive accuracy of a nonlinear model, can accurately characterize a large class of airborne and ground-based robotic systems. Specifically, a bilinear Koopman dynamic deep neural network (BKDDNN) is developed to learn the finite-dimensional bilinear Koopman operator in the lifting space without prior knowledge or system parameters. Moreover, the bilinear model is integrated into the standard model predictive control (MPC) optimization problem, facilitating the solution of the bilinear optimization problem. In such a way, the proposed DBKMPC avoids the problems of excessive inductive bias and selection difficulty of dictionary functions encountered by the existing methods, so that it enables a more effective solution to the problem of modeling and control of nonlinear robotic systems. The experimental results show that the proposed DBKMPC method surpasses the existing representative methods in terms of prediction and control performance.

Index Terms—Bilinear Koopman operator, model predictive control (MPC), neural network modeling.

I. INTRODUCTION

THE Koopman operator has emerged as a powerful tool for revealing nonlinear dynamics by expressing them as linear systems within an infinite-dimensional observable space [1]. This strategy allows established linear control theories to be applied for desired control outcomes [2]. In contrast to the complex optimization solving process involved in nonlinear control algorithms, linear methods are concise, efficient, computationally fast, and easy to implement in real time [3]. Consequently, leveraging the linear representation provided by the Koopman

Manuscript received 9 November 2023; revised 22 February 2024; accepted 1 April 2024. Date of publication 9 May 2024; date of current version 11 October 2024. This work was supported in part by the National Natural Science Foundation of China under Grant U22B2040. (Corresponding author: Shi Yan.)

Dongdong Zhao, Boyu Li, Fuxiang Lu, and Shi Yan are with the School of Information Science and Engineering, Lanzhou University, Lanzhou 73000, China (e-mail: ddzhao@lzu.edu.cn; lufux@lzu.edu.cn; yanshi@lzu.edu.cn).

Jinhua She is with the School of Engineering, Tokyo University of Technology, Hachioji 152-8550, Japan (e-mail: she@stf.teu.ac.jp).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TIE.2024.3390717>.

Digital Object Identifier 10.1109/TIE.2024.3390717

framework enables extending traditional control approaches such as linear quadratic regulators (LQR) [4] and model predictive control (MPC) [5] to address intricate nonlinear dynamics. This extension has garnered significant attention across various engineering domains, including robotics [6], power systems [7], meteorological forecasts [8], and autonomous racing [9], [10].

However, the Koopman operator usually acts as an infinite-dimensional global linear operator. Thus, it is crucial to obtain a finite-dimensional approximation of this operator for practical applications. Approximation approaches mainly fall into two categories: data-driven methods [2], [11] and deep neural networks (DNNs) [12], [13]. Data-driven methods often construct observable functions using customized function dictionaries, such as polynomial functions and radial basis functions. However, the choice of basis functions may limit model accuracy, and it is difficult to determine the appropriate basis function for unknown systems [14]. On the other hand, with the rise of deep learning, using DNNs to identify finite Koopman representations has become an appealing data-driven option, eliminating the need for selecting function dictionaries and allowing observables to be learned directly from data [15].

Importantly, while the Koopman operator is linear in terms of observables, it may not be compatible with computationally efficient linear control designs such as LQR and linear MPC when considering control inputs [16]. In practice, many robotic systems incorporate input forces that affect dynamics through rotation matrices, leading to nonlinear control inputs [17]. To address this challenge, bilinear models offer a balance between nonlinear inputs and linear control within the Koopman framework, retaining computational benefits while accommodating diverse dynamics in various systems. Notably, recent studies suggest that each control-affine system can be represented as a bilinear system, although it is not necessarily linear. The enhanced dynamic mode decomposition (EDMD) algorithm for linear modelling is extended to bilinear modelling by constructing enhanced bilinear models by least squares [16]. Another approach involves constructing an improved bilinear model based on the Koopman eigenfunction set using the Koopman canonical transform (KCT) [18]. Both methods utilize customized function dictionaries, ensuring ease of implementation and computational efficiency. These techniques have been employed to approximate bilinear models for planar quadrotors [18]. Nonetheless, selecting appropriate observable functions greatly influences prediction and control performance and is

challenging without prior system knowledge. To address this, the Koopnet DNN approach was proposed [19]. It eliminates the need to choose a function dictionary and can be effectively applied to a physical planar quadrotor system. However, this method enforces certain inductive biases, such as fixing the derivative relationship between states, which limits its generality to unknown systems.

To overcome the inherent limitations of the existing methods such as inductive bias and dictionary functions and provide a more accurate and effective method for the control of nonlinear dynamic systems, this article presents a novel approach, deep bilinear Koopman model predictive control (DBKMPC), for nonlinear systems. The contributions encompass two pivotal aspects.

- 1) First, a bilinear Koopman dynamic deep neural network (BKDDNN) is developed. This network facilitates the establishment of highly accurate bilinear models, even when dealing with unknown or poorly understood nonlinear dynamical systems. The training loss function is given in terms of the general laws and controllability of the bilinear model, without enforcing a special inductive bias, and is applicable to most systems.
- 2) Second, leveraging the strengths of the refined bilinear model, bilinear Koopman MPC is introduced and the constrained transformed square formula for the bilinear model is derived. This control scheme delivers computational efficiency comparable to linear models while effectively addressing optimal control problems that feature both input and state constraints.

The article is structured as follows. Some preliminaries are presented in the next section. The DBKMPC scheme is presented in Section III. Section IV provides a comparison of the proposed method with other approaches on a robotic system. Conclusions are provided in Section V.

II. PRELIMINARIES

A. Nonlinear Systems

Consider a nonlinear dynamical system described by the discrete-time model [19]

$$\mathbf{x}^+ = \mathbf{F}(\mathbf{x}, \mathbf{u}) \quad (1)$$

where $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{u} \in \mathbb{R}^m$ are the state vector and the input vector of the system, respectively, and $\mathbf{x}^+$ is the successor state. The function $\mathbf{F}$ represents the transition mapping. In the following discussion, $\mathbf{x}_k$ and $\mathbf{u}_k$ refer to the state and control action applied to the system at sampling time k . The system is subject to constraints on both the states and control actions given by [19]

$$\mathbf{x} \in \mathcal{X}, \quad \mathbf{u} \in \mathcal{U} \quad (2)$$

where $\mathcal{X}$ is a closed set and $\mathcal{U}$ is a compact set, both of which contain the origin. Let $\phi: \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X} \times \mathcal{U}$ denote the flow map of the system. The solution $\mathbf{x}_t$ of the system for a given sequence of control inputs $\mathbf{U}$ and initial state $\mathbf{x}$ can be represented as $(\mathbf{x}_t, \mathbf{U}) = \phi_t(\mathbf{x}, \mathbf{U})$ [16].

MPC is an iterative optimization strategy that solves an optimization problem to obtain control inputs for each time step of a closed-loop operation [20], and it can be used to control the above nonlinear system. At the present time k , let M be the prediction horizon and $\mathbf{x}_{t,\text{des}}$ denote the expected state at the point $t = k + 1, \dots, k + M$. For system (1), the following cost function is usually formulated for computing the optimal controls over the prediction horizon [19]

$$J_k = (\mathbf{x}_{k+M} - \mathbf{x}_{k+M,\text{des}})^\top P (\mathbf{x}_{k+M} - \mathbf{x}_{k+M,\text{des}}) + \sum_{t=k}^{k+M-1} (\mathbf{x}_t - \mathbf{x}_{t,\text{des}})^\top Q (\mathbf{x}_t - \mathbf{x}_{t,\text{des}}) + \mathbf{u}_t^\top R \mathbf{u}_t. \quad (3)$$

Here, $Q \in \mathbb{R}^{n \times n}$ denotes the state-related gain matrix, $P \in \mathbb{R}^{n \times n}$ denotes the final state-related gain matrix, and $R \in \mathbb{R}^{m \times m}$ denotes the control input-related gain matrix. The boundary constraints (2) and the following linear constraints are naturally derived:

$$G\mathbf{x} < \mathbf{d}. \quad (4)$$

Here, $G \in \mathbb{R}^{n \times n}$ denotes the coefficient matrix, and $\mathbf{d} \in \mathbb{R}^{n \times 1}$ denotes the vector consisting of boundary values.

The optimization problem in (3) defines a feedback controller, i.e.,

$$K_M(\mathbf{x}_k) = \mathbf{U}(\mathbf{x}_k) = \{\mathbf{u}_k(\mathbf{x}_k), \mathbf{u}_{k+1}(\mathbf{x}_k), \dots, \mathbf{u}_{k+M-1}(\mathbf{x}_k)\} \quad (5)$$

where $\mathbf{u}_t(\mathbf{x}_k)$, $t = k, \dots, k + M - 1$, denotes an optimal solution to the problem in (3) parameterized by the current state $\mathbf{x}_k$. Then, only the first value $\mathbf{u}_k(\mathbf{x}_k)$ in the optimal control input sequence $\mathbf{U}(\mathbf{x}_k)$ is fed into the system to obtain the succeeding state $\mathbf{x}_{k+1}$. Similarly, one can obtain an optimal solution $\mathbf{U}(\mathbf{x}_{k+1})$ to the problem in (3) parameterized by the state $\mathbf{x}_{k+1}$. In summary, the concerned MPC for the nonlinear systems can be stated as follows.

Problem 1: For the nonlinear dynamical system in (1), given the expected state $\mathbf{x}_{t,\text{des}}$, a state feedback tracking MPC law $\mathbf{u} = K_M(\mathbf{x})$ that minimizes the cost function in (3) with the constraints in (2) and (4) is designed.

B. Koopman Operator Theory

This subsection introduces the Koopman operator for the dynamical system in (1). The Koopman operator $\mathcal{K}$ is defined with respect to the flow map of the system and satisfies the following equation:

$$\mathcal{K}f = f \circ \phi. \quad (6)$$

Here, $f \in \mathcal{F}$ is referred to as an observable function, and $\mathcal{F}$ is an infinite-dimensional function space consisting of square-integrable real-valued functions with a compact domain $\mathcal{X} \times \mathcal{U} \subset \mathbb{R}^{n \times m}$. The Koopman operator $\mathcal{K}: \mathcal{F} \rightarrow \mathcal{F}$ is a linear operator acting on the observables. The symbol $\circ$ denotes the composition of functions, defined as

$$(f \circ \phi)(\mathbf{x}_k, \mathbf{u}_k) = f(\phi(\mathbf{x}_k, \mathbf{u}_k)) \quad (7)$$

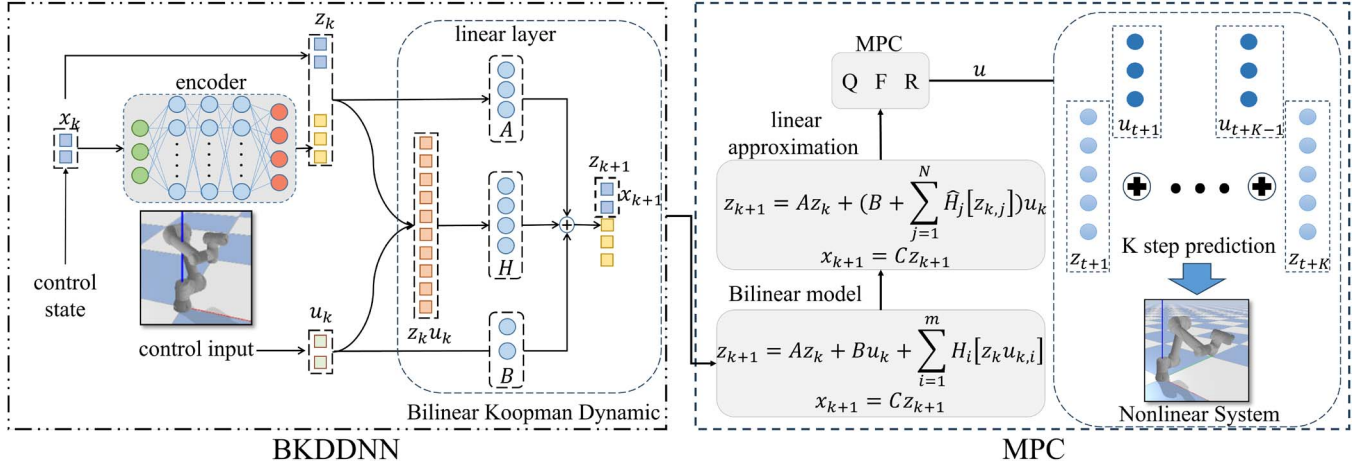


Fig. 1. Outline of the proposed DBKMPC methodology for robotic systems.

which leads to the following relationship:

$$(\mathcal{K}f)(x_k, u_k) = f(x_{k+1}, u_k). \quad (8)$$

Importantly, the input u_k is considered constant over time interval $[k, k+1]$. For a given time step, $\mathcal{K}$ is referred to as the discrete-time Koopman operator.

Due to its infinite-dimensional nature, the Koopman operator is often used in conjunction with a finite-dimensional approximation [18]. This approximation involves identifying a countable set of key observables $\{f_i \in \mathcal{F}\}_{i=1}^N$ with $N \in \mathbb{Z}$. If the observables contain all the state components of the original system, then the original state can be recovered easily.

C. Bilinear Model Realization

Considering the bilinear model realization in Section II-C, the goal is to establish a dynamical system that generates the same state response x as (1), given any input signal u and current state. In this article, the Koopman operator is leveraged to develop a bilinear model of the dynamical system.

To establish this bilinear model, a set of observables $\{f_i \in \mathcal{F}\}_{i=1}^N$ is utilized to form the lifted state of the realization. The evolution of each component of this lifted state is described by applying the discrete Koopman operator. The output equation of the realization involves the mapping C , and the model realization is expressed as

$$f_i(k+1) = \mathcal{K}f_i(k) \quad \text{for } k = 0, 1, \dots, N \quad (9)$$

$$x_k = C(f_1(k), \dots, f_N(k)). \quad (10)$$

Here, $f_i(k)$ is shorthand for $f_i(x_k, u_k)$. This realization is defined over a set of observables, and its specific form depends on the choice of observables. Importantly, since the Koopman operator is linear, a realization generated using the Koopman operator is linear with respect to the observables over which it is defined. To obtain the bilinear model, the following subsets of the space of all observables are defined as

$$\mathcal{X} := \{f \in \mathcal{F} | f(\tilde{x}, \tilde{u}) = \tilde{x}_i \text{ for some } i = 1, \dots, n\}, \quad (11)$$

$$\mathcal{U} := \{f \in \mathcal{F} | f(\tilde{x}, \tilde{u}) = \tilde{u}_i \text{ for some } i = 1, \dots, m\}, \quad (12)$$

$$\mathcal{Z} := \{f \in \mathcal{F} | f(\tilde{x}, \tilde{u}_1) = f(\tilde{x}, \tilde{u}_2), \forall \tilde{u}_1, \tilde{u}_2 \in \mathbb{R}^m\}. \quad (13)$$

Here, $\mathcal{Z}$ is the set of functions that exclusively depend on the state. The construction of the bilinear model can be realized using a set of observables $\{z_i \in \mathcal{Z}\}_{i=1}^N$. Based on these sets of functions, the bilinear model realization is defined as follows.

Definition 1 (Bilinear model realizations): For the system in (1), choose a set of observables $\{z_i \in \mathcal{Z}\}_{i=1}^N$. If there exist coefficient matrices $A \in \mathbb{R}^{N \times N}$, $B \in \mathbb{R}^{N \times m}$, $H_i \in \mathbb{R}^{N \times N}$ and $C \in \mathbb{R}^{N \times n}$ such that

$$z_{k+1} = Az_k + Bu_k + \sum_{i=1}^m H_i[z_k u_{k,i}] \quad (14)$$

and the original state can be obtained by mapping C , i.e.,

$$x_k = Cz_k \quad (15)$$

where $z_k = [z_{k,1}, \dots, z_{k,N}]^\top$, $x_k = [x_{k,1}, \dots, x_{k,n}]^\top$ and $u_k = [u_{k,1}, \dots, u_{k,m}]^\top$, then the model realization is said to be bilinear. If all elements of $H_i, i = 1, \dots, m$ are 0, then the realization is said to be linear.

III. METHOD

In this section, a DBKMPC method is proposed for discrete-time dynamical systems. As shown in Fig. 1, DBKMPC consists of two main parts. First, a BKDDNN is developed to directly learn the observable function and construct a lifting bilinear model of the controlled nonlinear dynamical system. Second, based on the constructed bilinear lifting model, an MPC method considering real space constraints is proposed to solve the control problem with state constraints.

A. Bilinear Koopman Modeling With DNNs

1) Modeling Method: In this section, a modelling approach BKDDNN is developed with the aim of exploring the bilinear Koopman representation of nonlinear dynamical systems. This method exploits the powerful learning capability of DNNs for handling nonlinear functions. The DNN structure, illustrated in

Fig. 1, comprises two neural networks: an encoder and a linear fitting layer.

The encoder takes the current original state and generates the observable value in a high-dimensional space. This high-dimensional observable space encompasses the low-dimensional state-space. Concatenating the original state with the encoder output results in a group of state vectors, which form the lifted state in the Koopman-invariant subspace. Moreover, the original state can subsequently be restored by mapping using matrix C . The linear fitting layer fits a finite-dimensional optimal representation of the bilinear Koopman operator in lifting space to the data. A two-part neural network is considered as a whole for training and gradient descent optimization. The architecture of the encoder and linear fitting layer consists of DNNs with single or multiple hidden layers.

By tuning the strength of the sparsity constraint, the network can learn different systems. A linear model can be learnt by forcing the H_i matrix to be a zero-matrix. In general, the BKDDNN can learn models of systems where the inputs and states have coupling. For instance, for the following simple system:

$$\begin{aligned} \dot{x} &= x + ux, & x(0) &= 0 \\ y &= x \end{aligned} \quad (16)$$

consider three trajectories

$$\begin{aligned} u_1 &\equiv -1 \longrightarrow x_1 = 0 \\ u_2 &\equiv 1 \longrightarrow x_2 = e^{2t} - 1 \\ u_3 &\equiv 2 \longrightarrow x_3 = e^{3t} - 1. \end{aligned} \quad (17)$$

If the system (16) has a lifting linear model conforming to two trajectories x_2 and x_3 , then its prediction for x_1 is as follows:

$$\begin{aligned} u_1 &= u_2 - u_3 \\ x_{LTI}(t) &= x_2(t) - x_3(t) = e^{2t} - e^{3t} \end{aligned} \quad (18)$$

by superposition of linearity. It is clear that the predictions are very poor and therefore no accurate lifting linear model exists.

2) **Loss Function:** Training involves iteratively updating network parameters using several loss functions. The primary loss function is the K -step prediction loss function, which focuses on the weighted sum of K -step prediction losses to enhance the long-term prediction ability. The K -step prediction loss function is expressed as

$$L_1(\theta) = \sum_{k=0}^K \gamma^{k-1} \text{MSE}(z_k, \hat{z}_k). \quad (19)$$

Here, γ serves as a weight attenuation parameter, θ incorporates the parameters of the encoder network and the linear fitting layer, and $\text{MSE}(z_k, \hat{z}_k)$ is the mean squared error loss function of the lifted state vector z_k and the predicted lifted state vector $\hat{z}_k$. The lifted state z_k is defined as

$$z_k = g(x_k) \quad (20)$$

where $g(\cdot)$ is the observable function of the Koopman-invariant subspace learned by the encoder under the parameters θ . The predicted lifted state can be written as

$$\hat{z}_k = A z_{k-1} + B u_{k-1} + \sum_{i=1}^m H_i [z_{k-1} u_{k-1,i}]. \quad (21)$$

The second loss function enforces mutual independence among states and is defined by

$$L_2(\theta) = \text{rank}(A) - N. \quad (22)$$

Here, $\text{rank}(A)$ represents the rank of control matrix A , and N is the lifting state dimension. A control matrix A with full rank indicates that all states are mutually independent and cannot be linearly expressed in relation to one another. This condition guarantees that the encoder-learned observable value functions are independent and effective representations of the state, reducing resemblance and redundancy in state functions.

The third loss function enforces sparsity in the bilinear term coefficient matrix H_i and is defined by

$$L_3(\theta) = \|H_i\|_1. \quad (23)$$

H_i represents the coefficient matrix of the bilinear term and serves as the parameterized bilinear Koopman operator matrix through a linear fitting layer. Sparsity constraints are needed to ensure the controllability of the bilinear state equations at each step in the control algorithms, as failure to do so can lead to issues in tracking desired trajectories.

Combining the above three loss functions, the final loss function for training is as follows:

$$\text{Loss} = L_1(\theta) + \alpha L_2(\theta) + \beta L_3(\theta). \quad (24)$$

Here, α and β are weights assigned to the independence-constrained and sparsity-constrained loss functions, respectively, which are given empirically. The $L_1(\theta)$ loss function represents the error between the true linear evolution of the system in the Koopman-invariant subspace and the model developed. The $L_2(\theta)$ loss function, from the control theory point of view, ensures that the neural network does not repeatedly learn two state functions that are close to each other, by constraining the rank of the control matrix of the state-space model to make each state function in the ascending space linearly uncorrelated. Generally, the control input $u_{k,1}, \dots, u_{k,m}$ is not couple to each state $z_{k,1}, \dots, z_{k,N}$, so the H_i matrix in (23) is usually sparse. The $L_3(\theta)$ loss function constrains the H matrix to be a sparse matrix from this law [21], [22]. The BKDDNN is trained using gradient descent via (24). The process of BKDDNN is summarized in Algorithm 1.

B. Bilinear Koopman Model Predictive Control

In this section, based on the bilinear model constructed in (21), the optimal control problem for a nonlinear system is transformed into that of bilinear system. The objective function

Algorithm 1 DBKMPC Modeling (BKDDNN)

Require: Dataset $(\mathbf{x}_k, \mathbf{u}_k, \mathbf{x}_{k+1})_{k=1, \dots, \text{len}}^{1, \dots, \text{num}}$;

- 1: Initialise network parameters $g(\cdot)$, A , B , H_i .
- 2: Set iterations p and training epochs q .
- 3: **for** each i in $1, \dots, q$ **do**
- 4: Randomly disrupt the order of the dataset.
- 5: **for** each j in $1, \dots, p$ **do**
- 6: Obtain the state $\mathbf{z}_k = g(\mathbf{x}_k)$, $\mathbf{z}_{k+1} = g(\mathbf{x}_{k+1})$.
- 7: Get the state $\hat{\mathbf{z}}_{k+1}$ by (14).
- 8: Calculate Loss funtion by (24).
- 9: Optimise parameters $g(\cdot)$, A , B , H_i by Loss.
- 10: **end for**
- 11: **end for**
- 12: **return** $g(\cdot)$, A , B , H_i .

is as follows:

$$\begin{aligned} \min J_k &= (\mathbf{z}_{k+M} - \mathbf{z}_{k+M, \text{des}})^\top P (\mathbf{z}_{k+M} - \mathbf{z}_{k+M, \text{des}}) \\ &+ \sum_{t=k}^{k+M-1} (\mathbf{z}_t - \mathbf{z}_{t, \text{des}})^\top Q (\mathbf{z}_t - \mathbf{z}_{t, \text{des}}) + \mathbf{u}_t^\top R \mathbf{u}_t \end{aligned} \quad (25)$$

subject to

$$\begin{aligned} \mathbf{z}_{k+1} &= A\mathbf{z}_k + B\mathbf{u}_k + \sum_{i=1}^m H_i [\mathbf{z}_k \mathbf{u}_{k,i}] \\ \mathbf{x} &\in \mathcal{X}, \mathbf{u} \in \mathcal{U}, \\ G\mathbf{x} &< d, \quad \mathbf{z}_0 = g(\mathbf{x}_{0, \text{des}}). \end{aligned} \quad (26)$$

Problem 2: For bilinear model (21) of system (1), given the expected state $\mathbf{x}_{t, \text{des}}$, a state feedback tracking MPC law $\mathbf{u} = K_M(\mathbf{x})$ that minimizes the cost function in (25) with the constraints in (26) is designed.

However, due to the inherent nonlinearity of bilinear systems, Problem 2 cannot be guaranteed to have a real-time solution and a global optimal solution. The equality constraint of the bilinear model makes the problem nonconvex. Linear approximation is required by fixing the value of the lifting state in the bilinear terms and redividing the coefficient matrix. The equality constraint in the MPC problem becomes the following:

$$\begin{aligned} \mathbf{z}_{k+1} &= A\mathbf{z}_k + B\mathbf{u}_k + \sum_{i=1}^m H_i [\mathbf{z}_k \mathbf{u}_{k,i}] \\ &= A\mathbf{z}_k + B\mathbf{u}_k + \sum_{i=1}^m \left[\sum_{j=1}^N \mathbf{h}_{ij} \mathbf{z}_{k,j} \mathbf{u}_{k,i} \right] \\ &= A\mathbf{z}_k + B\mathbf{u}_k + \sum_{j=1}^N \left[\sum_{i=1}^m \mathbf{h}_{ij} \mathbf{z}_{k,j} \mathbf{u}_{k,i} \right] \\ &= A\mathbf{z}_k + B\mathbf{u}_k + \sum_{j=1}^N \hat{H}_j [\mathbf{z}_{k,j}] \mathbf{u}_k \\ &= A\mathbf{z}_k + \left(B + \sum_{j=1}^N \hat{H}_j [\mathbf{z}_{k,j}] \right) \mathbf{u}_k \end{aligned} \quad (27)$$

with $\hat{H}_j = [\mathbf{h}_{1j} \ \dots \ \mathbf{h}_{mj}]$, $H_i = [\mathbf{h}_{i1} \ \dots \ \mathbf{h}_{iN}]$. At the current moment k , $[\mathbf{z}_{k,j}]_{j=1}^N$ is the initial lifting state. Separating the input term $\mathbf{u}$ from the bilinear part, along with the previous term, transforms the equation into a form that resembles the input linear. The resulting linear dynamics represents the behavior of the bilinear realization in the neighborhood of $[\mathbf{z}_{k,j}]_{j=1}^N$. Although this introduces prediction error, it can be solved in real time by transforming it into a convex quadratic programming problem, providing a unique global optimal solution that can be updated at each time step.

For the linearized bilinear system, the constraints of (26) in MPC becomes the following:

$$\begin{aligned} \mathbf{z}_{k+1} &= A\mathbf{z}_k + \left(B + \sum_{j=1}^N \hat{H}_j [\mathbf{z}_{k,j}] \right) \mathbf{u}_k \\ \mathbf{x} &\in \mathcal{X}, \mathbf{u} \in \mathcal{U}, \\ G\mathbf{x} &< d, \quad \mathbf{z}_0 = g(\mathbf{x}_{0, \text{des}}). \end{aligned} \quad (28)$$

Based on the above discussion, Problem 1 can be transformed into Problem 3, which can be solved as a quadratic programming problem through equality constraints. Accordingly, the inequality constraints of Problem 1 are transformed into input linear constraints in the quadratic programming solution.

Problem 3: For linearized bilinear model (27) of system (1), given the expected state $\mathbf{x}_{t, \text{des}}$, a state feedback tracking MPC law $\mathbf{u} = K_M(\mathbf{x})$ that minimizes the cost function in (25) with the constraints in (28) is designed.

Assuming that the optimization interval is rolled over to the current moment $\mathbf{x}_k$, at which point the future state of the system is predicted for M steps, it is assumed that the future state of the system must satisfy the following constraints.

$$G\mathbf{x}_{k+1} < d. \quad (29)$$

The observables $\mathbf{z}_k$ consists of the original state $\mathbf{x}_k$ spliced with the encoded state so that the constraints of a state of $\mathbf{x}_k$ are equal to the constraints of the corresponding state of $\mathbf{z}_k$. Thus, it can obtain the following:

$$\hat{G}\mathbf{z}_{k+1} < \hat{d} \quad (30)$$

with $\hat{G} = [G, \mathbf{0}]$, $\hat{d} = [d, \mathbf{0}]^\top$.

It follows from (27) that

$$\begin{aligned} \hat{G} \left(A\mathbf{z}_k + \left(B + \sum_{j=1}^N \hat{H}_j [\mathbf{z}_{k,j}] \right) \mathbf{u}_k \right) &< \hat{d}, \\ \hat{G} \left(B + \sum_{j=1}^N \hat{H}_j [\mathbf{z}_{k,j}] \right) \mathbf{u}_k &< \hat{d} - \hat{G}A\mathbf{z}_k. \end{aligned} \quad (31)$$

It should be noted that (31) is the standard input constraint form for quadratic programming solutions. The detailed process of the proposed DBKMPC control approach is summarized in Algorithm 2. The flowchart of the proposed DBKMPC approach is summarized in Fig. 3.

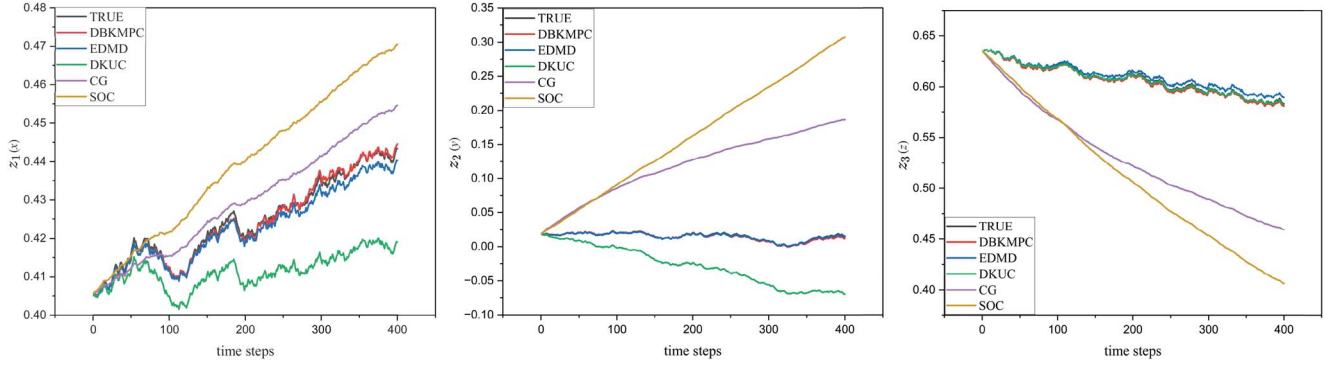


Fig. 2. 400-step prediction comparison chart of end-effector 3-D coordinates.

Algorithm 2 DBKMPC Control

Require: Initial state x_0 ; Desired trajectory $x_{k,desired}$; Encoder $g(\cdot)$; Bilinear model A, B, H_i, C ; Nonlinear system $F(x, u)$.

- 1: Set control steps T and prediction horizon M .
- 2: Initialise MPC parameters Q, R, F .
- 3: Encode state $z_0 = g(x_0)$, $z_{k,desired} = g(x_{k,desired})$.
- 4: **for** each k in $0, \dots, T-1$ **do**
- 5: Compute the state $\hat{z}_{k+1}, \dots, \hat{z}_{k+M}$ by (14).
- 6: Calculate $\hat{z}_t - z_{t,desired}$, $t = k, \dots, k+M$.
- 7: Convert (26) to (28) by applying (27).
- 8: Get $U(z_k)$ in (5) by solving (25) with constraints (28).
- 9: Obtain $x_{k+1} = F(x_k, u_k)$ by applying u_k in (5).
- 10: Encode state $z_{k+1} = g(x_{k+1})$.
- 11: **end for**

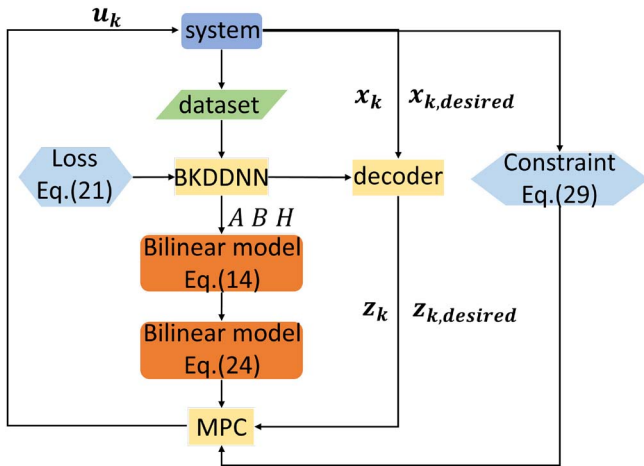


Fig. 3. Flowchart of DBKMPC.

IV. APPLICATION: CONTROL OF A ROBOTIC MANIPULATOR

In this section, the effectiveness of the proposed DBKMPC method will be verified by applying the UR5e robotic system. Physical experiments on trajectory tracking with obstacles were carried out on a UR5e robot and control and prediction accuracy

was compared with other control methods in a simulated environment. Notably, UR5e is a six-DOF manipulator, representing one of the most widely utilized manipulators.

A. Modelling and Prediction

1) *Data Collection and Model Identification:* The motion data are collected at 0.002 s intervals on the simulation platform to construct a dataset for establishing bilinear models of the UR5e robotic manipulator. Random inputs within the range of 0.3 to -0.3 are applied to the PID controller at the joint to generate random motion data. A total of 50 000 tracks for training and 20 000 tracks for verification are given in the simulation environment. Each track contains 31 data points, including the 3-D coordinates (x, y, z) of the end-effector, the angle, and the velocity of the robot's six joints, with the applied control being the joint velocity.

Next, the modelling and control methods of DBKMPC are used in the simulation environment to control the UR5e manipulator to track the desired state. The training is executed using an RTX3060 graphics card in a Python 3.9 environment under the PyTorch 1.10 framework on Windows 11. The loss during training is shown in Fig. 4. From the figure, it can be seen that the loss function is mainly affected by the $L_1(\theta)$, which converges after 250 000 epochs and obtains a good result. On the other hand, $L_2(\theta)$ and $L_3(\theta)$ quickly converge around the value of 0. The control methods are described in Section III-B, while the detailed parameters of modelling and control are shown in Table I.

In terms of prediction performance, the DBKMPC method was compared with the CG, SOC, DKUC, and EDMD methods. With the same initial conditions and running constants, predictions were made for these five types of models, choosing the 3-D coordinates of the end-effector and the six joint angles as the states to be predicted. Table II presents the prediction errors for 5-step, 50-step, 200-step, and 400-step prediction processes, demonstrating the short-term and long-term prediction capabilities of each method. The specific comparison chart of the prediction effect is shown in Fig. 2. It shows that in x and z coordinates, the model built by DBKMPC has a significant predictive advantage over the models of other methods. As well as, the position of the end-effector is determined by the

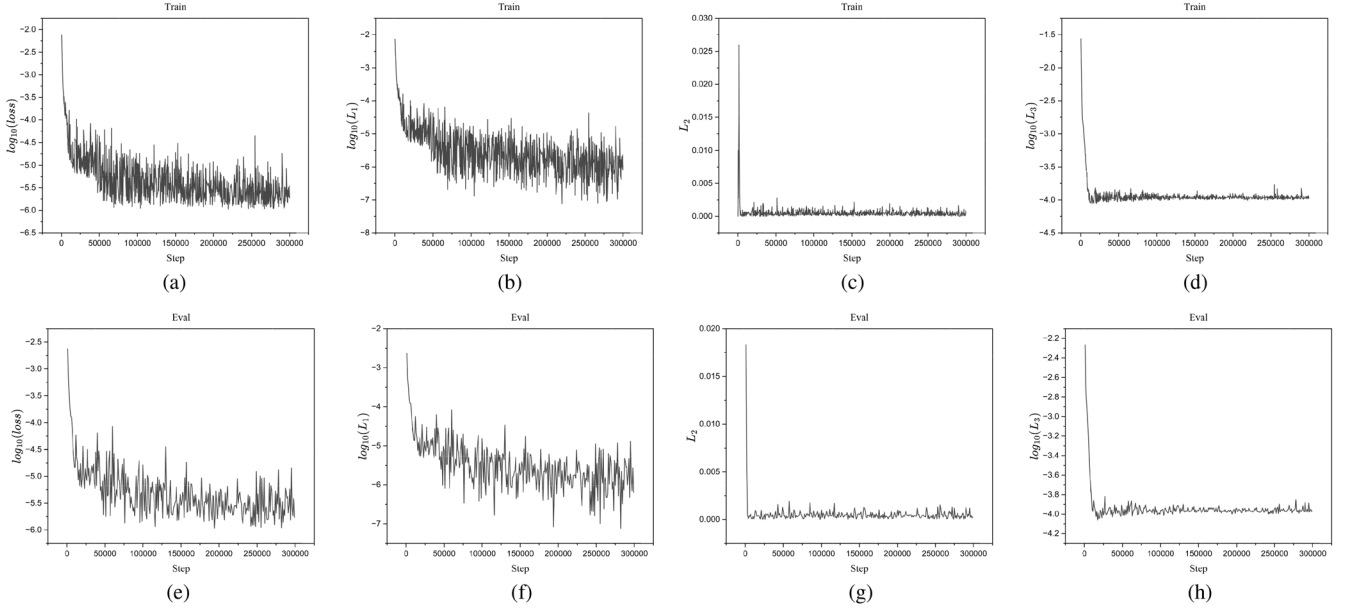


Fig. 4. Loss-descent plots during training and validation. (a) Train loss. (b) $L_1(\theta)$. (c) $L_2(\theta)$. (d) $L_3(\theta)$. (e) Eval loss. (f) $L_1(\theta)$. (g) $L_2(\theta)$. (h) $L_3(\theta)$.

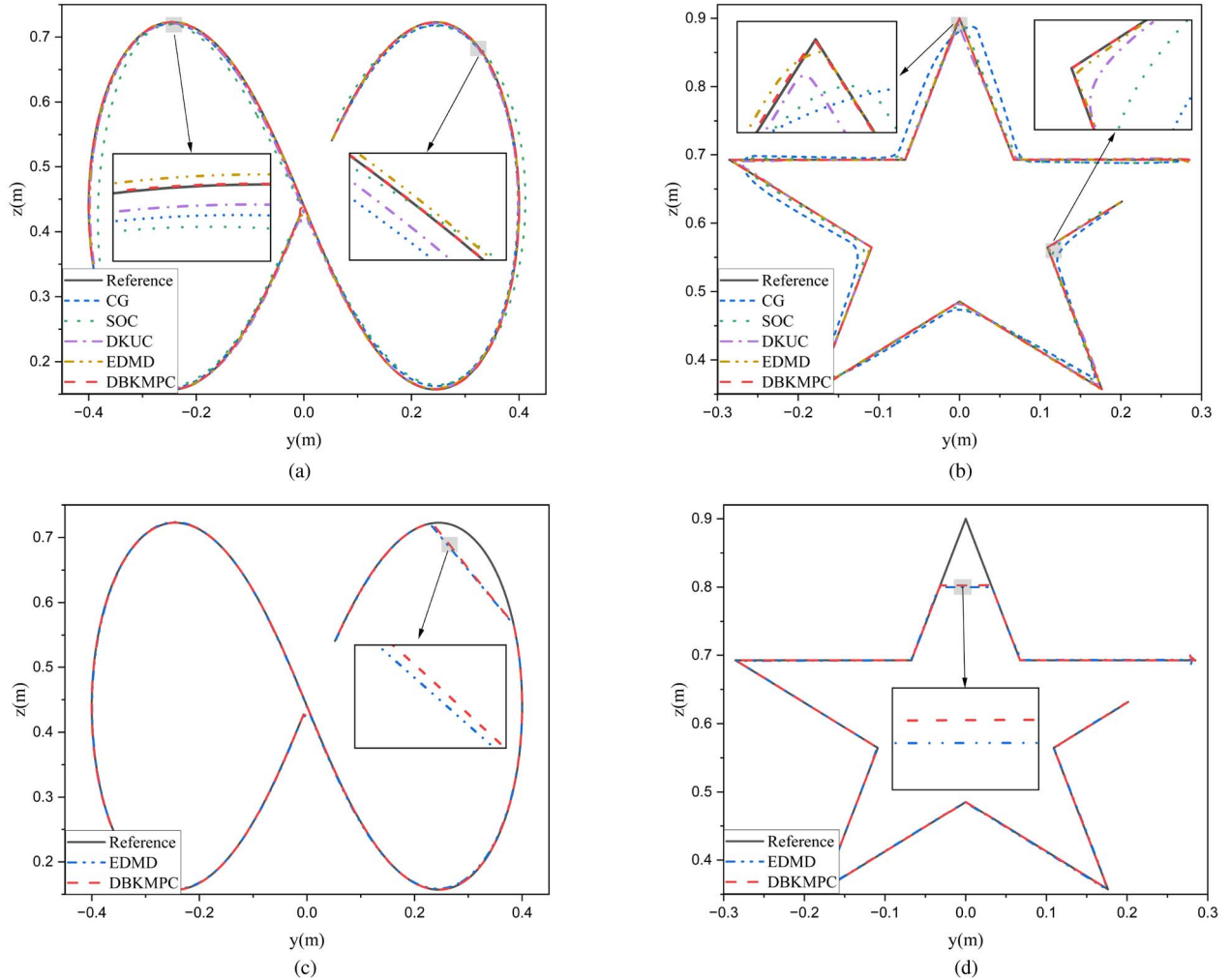


Fig. 5. Comparison of the control effect of the DBKMPC algorithm with the CG-LQR, SOC-LRQ, DKUC-LQR, and EDMD-MPC controllers on the UR5e robotic manipulator. The desired trajectory is the black line displayed on the $y - z$ plane. Top: LQR and MPC results. Bottom: MPC results with constraints. (a) UR5e-Lissajous. (b) UR5e-Star. (c) UR5e-Lissajous with constraint ($z + y \leq 0.95$). (d) UR5e-Star with constraint ($z \leq 0.8$).

TABLE I
EXPERIMENTAL PARAMETERS

Modelling		Control	
Parameters	Values	Parameters	Values
Layer structure	15 128 128 128 20	Q_{MPC}	$Q = 500 \times I_{35 \times 35}$
Activation function	Tanh	F_{MPC}	$F = 500 \times I_{35 \times 35}$
Optimizer	Adam	R_{MPC}	$R = 0.1 \times I_{6 \times 6}$
Learning rate	0.001	Q_{LQR}	$Q = I_{35 \times 35} \text{ or } 9 \times 9$
Epoch	300000	R_{LQR}	$R = 0.1 \times I_{6 \times 6}$
Batchsize	512	$kkt - solver$	ldl
α	1	$kktreg$	1^{-15}
β	0.1		
γ	0.8		

TABLE II
PREDICTION ERROR COMPARISON

Type	Method	Year	5-STEP	50-STEP	200-STEP	400-STEP
Linear	CG [23]	2007	0.1394%	0.7835%	2.7341%	4.9994%
	SOC [24]	2020	0.1570%	1.2893%	4.8175%	9.4575%
	DKUC [15]	2022	0.0285%	0.2401%	0.8792%	1.6921%
Bilinear	EDMD [16]	2021	0.0602%	0.1697%	0.4677%	0.5690%
	OURS	–	0.0129%	0.0498%	0.1569%	0.3265%

angles of the six joints of the robotic manipulator, so the 3-D coordinates of the end-effector can also reflect the prediction accuracy of each joint of the robotic manipulator. The prediction error is calculated as the sum of the two-norm of the difference between the predicted and actual trajectories for each trajectory point. It is divided by the sum of the two-norm of the actual trajectory, and then the square root is taken.

In the prediction experiments, the bilinear model obtained better prediction results than the linear model. The prediction error of this article's method is 0.0129%/0.0498%/0.1569%/0.3265% in the 5-step prediction, 50-step prediction, 200-step prediction, and 400-step prediction, respectively. Compared with the best available method, DBKMPC decreased the error by 0.0156%/0.1199%/0.3108%/0.2425% in the 5-step prediction, 50-step prediction, 200-step prediction, and 400-step prediction, respectively. The increase in long-term prediction error is also much smaller than that of the linear model, which proves that the bilinear structure is more compatible with real nonlinear dynamic systems.

B. Control Comparison and Physical Experiment

1) *Comparison With Model and Control Methods:* The CG-LQR controller, SOC-LQR controller, DKUC-LQR controller, EDMD-MPC controller, and DBKMPC controller are constructed using the models created by these five methods. These

TABLE III
CONTROL ERROR COMPARISON

Controller	Method	Year	Lissajous	Star	Lissajous ($z + y \leq 0.95$)	Star ($z \leq 0.8$)
LQR	CG [23]	2007	67.7897	60.8876	–	–
	SOC [24]	2020	79.9359	70.0110	–	–
	DKUC [15]	2022	53.2036	39.3168	–	–
MPC	EDMD [16]	2021	15.6089	13.9085	25.4909	41.8969
	OURS	–	8.3113	6.1296	23.0899	39.2867

controllers are used to simulate trajectories of Lissajous and Tu-Star patterns. Trajectories are generated in the y - z plane of the end-effector, and the joint angles are obtained through inverse kinematics to solve for the x , y , and z coordinates of the end-effector. Controllers compute control inputs by solving the optimal control problem in (25).

The performance of the five controllers is shown in Fig. 5(a) and 5(b). From the figures, it can be seen that DBKMPC is closer to the baseline than the other methods in the tracking of smooth trajectories. Even at the inflexion and cusp points, where the trajectory changes drastically, DBKMPC can track the trajectory well, while the other methods cannot do. The total tracking errors are summarized in Table III, in which the total tracking errors of DBKMPC on the Lissajous and star patterns are 8.3113/6.1296, which are 46.76%/55.93% lower than the best performing EDMD method.

2) *Control With Constraints:* Positional state constraints are added to the control process of the UR5e robotic manipulator, where $z + y \leq 0.95$ for tracking the Lissajous trajectory and $z \leq 0.8$ for tracking a star trajectory. LQR control cannot solve the optimal control problem of a robotic system with state constraints. Therefore, two MPC controllers are constructed to simulate tracking trajectory graphs with state constraints. The experimental results are presented in Fig. 5(c) and 5(d). DBKMPC has the ability to solve constrained optimization problems in a way that LQR does not, and the trajectories in the constrained domain fit more closely to the boundary lines. The total tracking errors of the control trajectories are provided in Table III.

The results show that DBKMPC can effectively solve the optimal control problem for robot systems with state constraints. Clearly, the bilinear model-based approach is much better than the linear model-based approach in terms of short-term and long-term prediction ability, while the total trajectory tracking error is lower than that of the linear modeling-based approach. The proposed DBKMPC method reduces the tracking error to less than 10 for the first time, which achieves an effect not found in other methods, while the tracking error with constraints is also lower than that of the EDMD method. It can be concluded that the proposed DBKMPC algorithm outperforms the CG, SOC, DKUC, and EDMD algorithms in both state-constrained and state-constraint-free robot modelling and control tasks. In order to further test the actual control effectiveness and stability of the method, physical experiments were carried out on the UR5e manipulator. As shown in Fig. 6, the manipulator

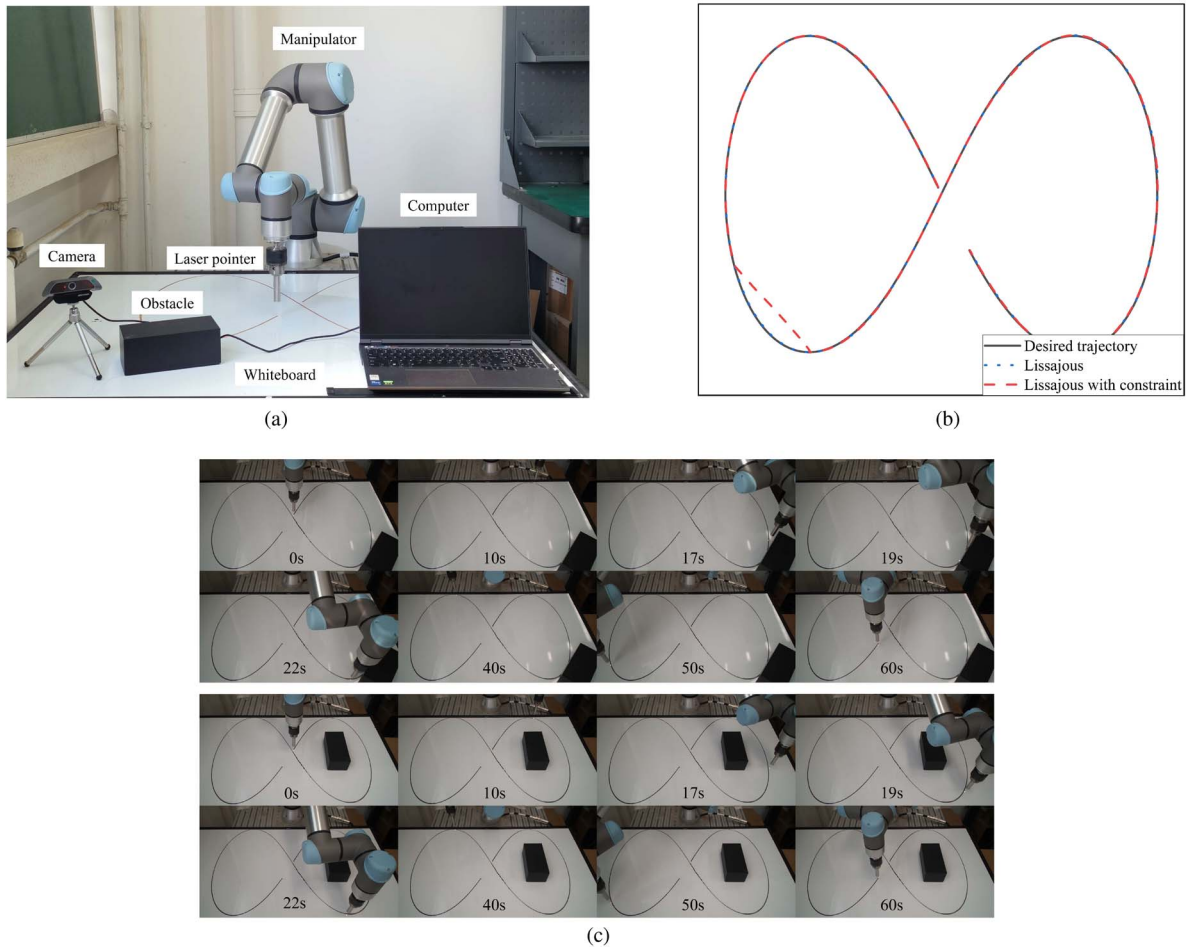


Fig. 6. (a) Physical experiment platform. (b) Control result. (c) Snapshots of the control process with constraints when tracking the Lissajous trajectory. The experiment videos are available on <https://www.youtube.com/watch?v=u8IT5UtHwYY&list=PLs-ilqifz7p1Jlpy5ygdB4eIFf1xeZiOZ>

successfully tracked a Lissajous curve in the horizontal plane and followed the desired trajectory.

V. CONCLUSION

In this article, a DBKMPC approach was developed for modelling and control of nonlinear systems. A BKDDNN was developed, which effectively learnt the finite-dimensional bilinear Koopman operator in the lifting space, thereby eliminating the need for any prior knowledge or system parameters. This significant advancement empowers the creation of highly accurate bilinear models tailored to nonlinear systems. Furthermore, the approach meticulously incorporated state terms that couple the system's input with its state, ensuring the precision of the bilinear models while maintaining system stability. To apply this approach, the generated bilinear model was seamlessly integrated into the standard MPC optimization problem, for streamlining the solution process of the bilinear optimization problems. The experimental results have provided compelling evidence of the superior predictive and control performance of the proposed DBKMPC method when compared with other existing representative methods. This research marks a substantial

step forward in the modelling and control of nonlinear systems and has great promise for a wide range of practical applications.

Future work should focus on the optimization of the state dimension in the bilinear model. This is paramount because a higher number of lifting states results in a significant increase on computation time, while a lower number of lifting state dimensions may reduce the prediction accuracy of the model.

REFERENCES

- [1] B. Lusch, J. N. Kutz, and S. L. Brunton, "Deep learning for universal linear embeddings of nonlinear dynamics," *Nature Commun.*, vol. 9, no. 1, 2018, Art. no. 4950.
- [2] H. Wang, W. Liang, B. Liang, H. Ren, Z. Du, and Y. Wu, "Robust position control of a continuum manipulator based on selective approach and Koopman operator," *IEEE Trans. Ind. Electron.*, vol. 70, no. 12, pp. 12522–12532, Dec. 2023.
- [3] G. Mamakoukas, M. L. Castano, X. Tan, and T. D. Murphey, "Derivative-based Koopman operators for real-time control of robotic systems," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 2173–2192, Dec. 2021.
- [4] E. Spyros-Papastavridis and J. S. Dai, "Stable flexible-joint floating-base robot balancing and locomotion via variable impedance control," *IEEE Trans. Ind. Electron.*, vol. 70, no. 3, pp. 2748–2758, Mar. 2023.
- [5] P. Qi, C. Liu, A. Ataka, H.-K. Lam, and K. Althoefer, "Kinematic control of continuum manipulators using a fuzzy-model-based approach," *IEEE Trans. Ind. Electron.*, vol. 63, no. 8, pp. 5022–5035, Aug. 2016.

- [6] D. Zhao, X. Yang, Y. Li, L. Xu, J. She, and S. Yan, "A Kalman–Koopman LQR control approach to robotic systems," *IEEE Trans. Ind. Electron.*, early access, Apr. 19, 2024, doi: 10.1109/TIE.2024.3379674.
- [7] Y. Xu, Q. Wang, L. Mili, Z. Zheng, W. Gu, S. Lu, and Z. Wu, "A data-driven Koopman approach for power system nonlinear dynamic observability analysis," *IEEE Trans. Power Syst.*, vol. 39, no. 2, pp. 4090–4104, Mar. 2024.
- [8] N. Geneva and N. Zabarar, "Transformers for modeling physical systems," *Neural Netw.*, vol. 146, pp. 272–289, Feb. 2022.
- [9] R. Wang, Y. Han, and U. Vaidya, "Deep Koopman data-driven optimal control framework for autonomous racing," *EasyChair Preprint*, vol. 5, pp. 1–8, 2021.
- [10] Y. Xiao, X. Zhang, X. Xu, X. Liu, and J. Liu, "Deep neural networks with Koopman operators for modeling and control of autonomous vehicles," *IEEE Trans. Intell. Veh.*, vol. 8, no. 1, pp. 135–146, 2022.
- [11] I. Abraham and T. D. Murphey, "Active learning of dynamics for data-driven control using Koopman operators," *IEEE Trans. Robot.*, vol. 35, no. 5, pp. 1071–1083, Oct. 2019.
- [12] M. Wang, X. Lou, W. Wu, and B. Cui, "Koopman-based MPC with learned dynamics: Hierarchical neural network approach," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 3630–3639, Mar. 2024.
- [13] J. Zhang and H. Wang, "Online model predictive control of robot manipulator with structured deep Koopman model," *IEEE Robot. Automat. Lett.*, vol. 8, no. 5, pp. 3102–3109, May 2023.
- [14] Z. C. Guo, V. Korotkine, J. R. Forbes, and T. D. Barfoot, "Koopman linearization for data-driven batch state estimation of control-affine systems," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 866–873, Apr. 2022.
- [15] H. Shi and M. Q.-H. Meng, "Deep Koopman operator with control for nonlinear systems," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 7700–7707, Jul. 2022.
- [16] D. Bruder, X. Fu, and R. Vasudevan, "Advantages of bilinear Koopman realizations for the modeling and control of systems with unknown dynamics," *IEEE Robot. Automat. Lett.*, vol. 6, no. 3, pp. 4369–4376, Jul. 2021.
- [17] S. Lian et al., "Adaptive attitude control of a quadrotor using fast nonsingular terminal sliding mode," *IEEE Trans. Ind. Electron.*, vol. 69, no. 2, pp. 1597–1607, Feb. 2022.
- [18] C. Folkestad and J. W. Burdick, "Koopman NMPC: Koopman-based learning and nonlinear model predictive control of control-affine systems," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 7350–7356.
- [19] D. Limón, T. Alamo, F. Salas, and E. F. Camacho, "On the stability of constrained MPC without terminal constraint," *IEEE Trans. Autom. Control*, vol. 51, no. 5, pp. 832–836, May 2006.
- [20] D. J. Kim, Y. W. Jeong, and C. C. Chung, "Lateral vehicle trajectory planning using a model predictive control scheme for an automated perpendicular parking system," *IEEE Trans. Ind. Electron.*, vol. 70, no. 2, pp. 1820–1829, Feb. 2023.
- [21] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proc. Nat. Acad. Sci.*, vol. 113, no. 15, pp. 3932–3937, 2016.
- [22] R. Strässer, J. Berberich, and F. Allgöwer, "Robust data-driven control for nonlinear systems using the Koopman operator," 2023, *arXiv:2304.03519*.
- [23] B. Boots, G. J. Gordon, and S. Siddiqi, "A constraint generation approach to learning stable linear dynamical systems," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 20, 2007, pp. 1329–1336.
- [24] G. Mamakoukas, O. Xherija, and T. Murphey, "Memory-efficient learning of stable linear dynamical systems for prediction and control," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 13527–13538.



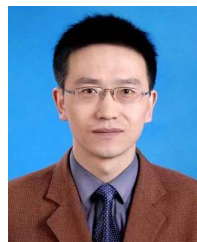
Dongdong Zhao (Member, IEEE) received the B.Eng. degree in engineering from Zhengzhou University, Zhengzhou, China, in 2012, the M.Eng. degree in engineering from Lanzhou University, Lanzhou, China, in 2016, and the Dr.Eng. degree in engineering from Akita Prefectural University, Akita, Japan, in 2019.

He is currently an Associate Professor with the School of Information Science and Engineering, Lanzhou University. His research interests include uncertain dynamical systems, robotics, machine learning, and multidimensional system theory.



Boyu Li received the B.Eng. degree in automation from Northeast University, Shenyang, China, in 2022. He is currently working toward the M.Eng. degree in communication engineering with the School of Information Science and Engineering, Lanzhou University, Lanzhou, China.

His current research interests include robotics and deep learning.



Fuxiang Lu received the Ph.D. degree in engineering from Shanghai Jiao Tong University, Shanghai, China, in 2012.

He is currently an Associate Professor with the School of Information Science and Engineering, Lanzhou University, Lanzhou, China. His research interests include machine learning, image processing, computer vision, and pattern recognition.



Jinhua She (Fellow, IEEE) received the B.S. degree from Central South University, Changsha, China, in 1983, and the M.S. and Ph.D. degrees from Tokyo Institute of Technology, Tokyo, Japan, in 1990 and 1993, respectively, all in engineering.

In 1993, he joined the School of Engineering, Tokyo University of Technology, Tokyo, where he is currently a Professor. His research interests include the applications of advanced control theory, repetitive control, process control, mobile

learning, and assistive robotics.

Dr. She is a Member of the Society of Instrument and Control Engineers, the Institute of Electrical Engineers of Japan, the Japan Society of Mechanical Engineers, Architectural Institute of Japan, and the Asian Control Association. He serves as a Technical Editor of *IEEE/ASME TRANSACTIONS ON MECHATRONICS* and an Associate Editor of *IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS*, *IEEE/CAA JOURNAL OF AUTOMATICA SINICA*, *IEEE JOURNAL OF EMERGING AND SELECTED TOPICS IN INDUSTRIAL ELECTRONICS*, *IEEE Journal of Industry Applications*, *Journal of Advanced Computational Intelligence and Intelligent Informatics*, *Intelligence and Robotics*, etc.



Shi Yan (Member, IEEE) received the B.S. and M.Eng. degrees in engineering from Lanzhou University, Lanzhou, China, in 2001 and 2004, respectively, and the Dr.Eng. degree in engineering from Akita Prefectural University, Akita, Japan, in 2010.

From 2004 to 2007, he was an Assistant Professor with the School of Information Science and Engineering, Lanzhou University. He was a Visiting Researcher, in 2007, and a Visiting Research Fellow, in 2010, with the Department

of Electronics and Information Systems, Akita Prefectural University. He was a Lecturer, from 2007 to 2012, and an Associate Professor, from 2012 to 2020, with the School of Information Science and Engineering, Lanzhou University, where he is currently a Professor. His research interests include multidimensional system theory, signal processing, and machine learning.